# TaxonVision-MLOps: Enterprise Strategy & Architecture

## High-Throughput Biodiversity Computer Vision, Mathematical Uncertainty Guarantees & Bare-Metal MLOps

Benjamin Förster

Liora Advanced Data Science & MLOps
Production Systems Architecture & Deep Learning Engineering

Production Architecture Presentation

# Curriculum Overview

## Structure of the Presentation

- **Ch 1:** Problem & Operational Maturity
- **Ch 2:** Hardware Architecture & Topography
- **Ch 3:** Asymmetric Dual-Pipeline Paradigm
- **Ch 4:** Data Engineering & Storage (Parquet/DVC)
- **Ch 5:** Model Engineering & Bayesian Tuning
- **Ch 6:** Multimodal Knowledge Graph & Vision-RAG
- **Ch 7:** Epistemic Safety & Conformal Bounds
- **Ch 8:** CI/CD, Containers & Kubernetes (k3s)
- **Ch 9:** Observability, Drift & Active Learning
- **Ch 10:** Delivery Roadmap & Core Principles

# The Real-World Problem

## Extreme Fine-Grained Taxonomic Classification

### Slide Topic & Explanatory Focus

This slide introduces the fundamental real-world problem: automated species identification across thousands of biological classes. In standard university coursework or Kaggle competitions, machine learning models are typically trained on balanced, clean datasets like CIFAR-10 or ImageNet with distinct visual boundaries. In biological field monitoring, observations follow an extreme long-tail distribution where common species have thousands of images while endangered taxa have fewer than five. Furthermore, inter-class variance can be microscopic (e.g., subtle wing venation in insects or spore color in fungi), while intra-class variance is vast due to lighting, life stages, sexual dimorphism, and occluding foliage.

**Status: [Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide addresses the primary architectural failure mode for junior data scientists: attempting to download massive datasets directly to local development drives. Biological repositories such as GBIF (Global Biodiversity Information Facility) and iNaturalist host tens of millions of high-resolution images totaling over 50 terabytes. Downloading, uncompressing, and caching this volume of raw media locally saturates workstation NVMe storage, introduces hours of IO latency, and invalidates reproducibility across team members. Production MLOps mandates zero-copy streaming, content-addressable storage, and decoupling feature extraction from model experimentation.

**Status: [Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide covers data governance, licensing constraints, and ethical compliance in machine learning pipelines. Training models on publicly scraped web images without license verification creates severe intellectual property liability and prevents commercial or academic distribution. TaxonVision enforces automated pre-commit gates that inspect observation metadata against the DarwinCore standard, strictly admitting only CC0 (Public Domain), CC-BY (Attribution), and CC-BY-NC (Non-Commercial) licensed media, while automatically generating attribution manifests and rejecting unverified static assets.

**Status: [Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide establishes the theoretical framework of MLOps maturity according to Google Cloud and industry standards, contrasting Level 0 with Level 4. Level 0 represents manual, script-driven machine learning characterized by Jupyter notebooks, untracked artifacts, manual training runs, and disconnected deployment steps. Level 4 represents automated, continuous learning pipelines featuring automated CI/CD gates, reproducible feature stores, drift-triggered retraining, automated mathematical validation, and blue/green production deployment. TaxonVision achieves Level 3/4 operational maturity on bare-metal hardware.

**Status: [Implemented & Verified]**

# Physical Topography

## Dual-Node Architecture: flagship vs hive-mind

### Slide Topic & Explanatory Focus

This slide outlines the physical hardware topology of the project, explaining the separation between the developer workstation (`flagship`) and the dedicated 24/7 bare-metal server (`hive-mind`). Rather than relying on expensive, rented public cloud instances that incur continuous hourly costs and bandwidth egress fees, TaxonVision deploys on high-performance on-premise hardware. Developers write code, test micro-pipelines, and run unit tests on `flagship`, while continuous integration, container registries, Kubernetes orchestration, and heavy background training execute autonomously on `hive-mind`.

**Status: [Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide explains the challenges of modern heterogeneous processor architecture, specifically the Intel Core i7-14700K equipped with 8 Performance-Cores (16 threads) and 12 Efficient-Cores (12 threads). In multi-threaded Python pipelines, unmanaged OS thread scheduling can place heavy matrix operations or image decompression tasks onto low-clocked E-cores, causing massive thread synchronization stalls and thermal throttling. We explain the necessity of explicit thread pinning, asynchronous event loops, and CPU affinity isolation for high-throughput data pipelines.

**Status: [Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide examines the modern GPU accelerator powering the system: the NVIDIA GeForce RTX 5070 Ti based on the Blackwell architecture (SM 120) with 16 GB of high-speed GDDR7 VRAM. We explain the balance between memory bandwidth, FP16/BF16/INT8 Tensor Cores, and thermal constraints. In contrast to data center H100 cards with 80 GB of HBM3 memory, consumer/workstation accelerators have strict 16 GB physical limits, making disciplined memory budgeting, frozen backbones, and zero dynamic memory allocation mandatory.

**Status:** **[Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide details our Kubernetes GPU resource management strategy: 4-slot GPU time-slicing coupled with an invariant 70% VRAM allocation ceiling (11.2 GB per container). Under standard Kubernetes scheduling, a single pod requests the entire physical GPU, locking out all other workloads. By configuring the NVIDIA Container Toolkit for time-slicing, up to four distinct workloads (e.g., active learning inference, feature extraction, background testing, and web API) share the GPU safely. Enforcing a strict 11.2 GB cap prevents CUDA allocation fragmentation and eliminates catastrophic out-of-memory kernel panics.

**Status: [Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide addresses secure external access to bare-metal infrastructure without public IPv4 exposure or vulnerable port forwarding. In typical home lab or on-premise environments, opening router ports to the internet invites automated credential brute-forcing and DDoS attacks. TaxonVision uses a persistent Cloudflare Zero Trust Tunnel (`cloudflared`) daemon running inside the Kubernetes cluster. Outbound encrypted tunnels connect directly to Cloudflare edge nodes, routing verified operator traffic into our Traefik ingress controller with TLS termination and zero exposed host ports.

**Status:** **[Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide introduces the overarching architectural paradigm of TaxonVision: the asymmetric dual-pipeline decoupled architecture. Junior practitioners often attempt to train an end-to-end convolutional or vision transformer network that directly outputs species logits. When new species are discovered or taxonomic revisions occur, the entire multi-gigabyte backbone must be painfully retrained. TaxonVision cleanly decouples static visual representation (Pipeline 1: frozen foundation backbones) from dynamic taxonomic reasoning (Pipeline 2: multimodal knowledge graph and vector retrieval), enabling agile updates and sub-second classification.

**Status: [Implemented & Verified]**

# Pipeline 1

## Offline Representation & Linear Probe Training

### Slide Topic & Explanatory Focus

This slide details Pipeline 1: the offline representation and classification workflow. High-resolution raw images stored in remote S3 are processed in batch streams by frozen backbones (DINOv2, BioCLIP) to generate dense feature embeddings (768 or 1024 dimensions). These embeddings are serialized into content-addressable storage. Downstream, lightweight linear probing classifiers are trained with Class-Balanced Loss and Bayesian hyperparameter sweeps in seconds rather than hours, producing ONNX-quantized models ready for high-throughput serving.

**Status:** **[Implemented & Verified]**

# Pipeline 2

## Taxonomic Multimodal Knowledge Graph (MMKG)

### Slide Topic & Explanatory Focus

This slide explains Pipeline 2: the continuous taxonomic ingestion and Multimodal Knowledge Graph (MMKG) streaming pipeline. Biological classifications are hierarchical: Kingdom, Phylum, Class, Order, Family, Genus, Species. Pipeline 2 ingests DarwinCore metadata streams, builds a directed acyclic graph (DAG) of biological lineage, and indexes multimodal embeddings into a vector and graph database. When visual ambiguity arises, the system performs Vision-RAG (Retrieval-Augmented Generation) against verified exemplar specimens and anatomical descriptions to resolve close morphological ties.

**Status: [Target Architecture / Roadmap]**

## Slide Topic & Explanatory Focus

This slide examines the runtime convergence point where Pipeline 1 and Pipeline 2 meet to serve live inference requests under strict latency budgets (p95 $< 50$ ms). When an image upload hits the FastAPI endpoint, it enters the Fast-Path: ONNX Runtime evaluates the quantized linear probe and computes conformal uncertainty sets in $< 15$ ms. If conformal coverage produces a high-confidence singleton set, the result returns immediately. Only ambiguous predictions or out-of-distribution observations trigger asynchronous enrichment from Pipeline 2's knowledge graph.

**Status: [Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide introduces how TaxonVision solves the "50 TB storage trap"—the impossibility of storing multi-terabyte raw citizen-science image corpora locally on development workstations. Instead of downloading raw JPEGs, the architecture relies on zero-copy streaming from AWS Open Data / iNaturalist S3 mirrors indexed by lightweight Apache Parquet columnar manifests (`observations.parquet`). We explain the separation between immutable remote pixel assets and local structured metadata, enabling complete dataset versioning and rapid querying without exhausting workstation NVMe storage.

**Status: [Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide examines the schema of `observations.parquet` and the cryptographic validation mechanism. Inside each row of the manifest, the schema stores observation keys and URLs (`observation_uuid`, `taxon_id`, `image_url`), DarwinCore taxonomic ranks, GPS coordinates, observation month, open license codes, and crucially a SHA-256 or MD5 content validation hash. When the `S3Streamer` module streams an image buffer into RAM via zero-copy Apache Arrow record batches, it verifies this cryptographic hash on the fly before passing the tensor to the feature extraction backbone, preventing corrupted or altered images from entering pipelines.

**Status: [Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide details our zero-memory-bloat feature loading mechanism: 1:1 row alignment between `observations.parquet` and memory-mapped NumPy embedding arrays (`embeddings.npy`). When foundation backbones (BioCLIP, DINOv2) extract dense 768-dimensional feature vectors ($z \in \mathbb{R}^{768}$), vectors are serialized into disk-backed `np.memmap` arrays. Each row index in the Parquet manifest aligns exactly 1:1 with the corresponding row in `embeddings.npy`, allowing PyTorch dataloaders to query metadata by hash or taxon in Parquet and immediately slice matching feature embeddings from disk without loading the entire multi-gigabyte array into RAM.

**Status: [Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide explores how Data Version Control (DVC) tracks dataset state at the repository level. DVC computes a master MD5 checksum over the Parquet manifest file itself (yielding e.g., `f546997db8b542e87e4f006326d04aac.dir`). DVC writes this hash into a lightweight pointer file (`data/sample.dvc`). Git tracks and commits only this text pointer, while DVC pushes the actual heavy Parquet file to our DagsHub S3 remote storage. This guarantees complete experiment reproducibility and exact dataset-to-model lineage without bloating Git history.

**Status: [Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide examines the production implementation of batch feature extraction (`extract_features.py`). We discuss how the pipeline streams images from remote S3 storage, passes batches through the frozen visual transformer backbone, extracts L2-normalized dense embeddings, and writes structured Parquet/LMDB files partitioned by dataset split (train, val, test). We also highlight the `build_taxon_index_map` function that guarantees deterministic integer label mapping across all splits, preventing subtle class mismatch bugs during downstream training.

**Status: [Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide analyzes the mathematical and economic differences between full network fine-tuning and linear probing on pre-cached embeddings. Full fine-tuning computes gradients through all transformer layers, consuming up to 16 GB of VRAM, requiring hours of compute, and risking catastrophic forgetting of foundational representations. Linear probing freezes the backbone and trains only a convex or shallow classification head on cached embeddings. This reduces training time from hours to seconds, drops VRAM requirements below 1 GB, and eliminates gradient backpropagation through the heavy backbone.

**Status: [Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide delves into the mathematical formulation of Class-Balanced Loss (Cui et al., CVPR 2019) used in our training runner. Under extreme long-tail class distributions, standard cross-entropy loss is dominated by head classes, causing the model to completely ignore rare species. Naïve inverse-frequency weighting fails because redundant images in head classes provide overlapping information. Class-Balanced Loss introduces the concept of the "effective number of samples" $E_n = (1 - \beta^n)/(1 - \beta)$, scaling class loss weights proportionally to their marginal information content.

**Status: [Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide examines the optimization dynamics implemented in `src/taxon_vision/models/training/runner.py`. We discuss AdamW optimization, cosine learning rate schedules with initial linear warmup to prevent early gradient divergence, and an early stopping mechanism that monitors validation loss over a patience window. Crucially, the runner implements deterministic best-weight restoration: upon training completion, the model restores the exact checkpoint corresponding to the minimum validation loss rather than the potentially overfitted final epoch.

**Status: [Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide details our automated hyperparameter tuning strategy using Optuna. Rather than relying on brute-force grid search (exponential complexity) or unguided random search, Optuna employs the Tree-structured Parzen Estimator (TPE) algorithm to model the probability distribution of promising hyperparameter regions (learning rate, weight decay, loss $\beta$, dropout). To maximize hardware efficiency, our runner integrates the Asynchronous Successive Halving Algorithm (ASHA) pruner, aggressively terminating unpromising trials in early epochs to reallocate GPU cycles to top candidates.

**Status: [Implemented & Verified]**

# Evaluation Metrics

## Macro PR-AUC vs Single-Threshold F1 Score

### Slide Topic & Explanatory Focus

This slide explains why TaxonVision uses Macro Precision-Recall AUC (PR-AUC) as its primary model promotion gate instead of the standard F1-score. In extreme class-imbalanced datasets, F1-score is highly sensitive to an arbitrary static decision threshold (typically 0.5), which heavily penalizes rare classes whose optimal threshold is much lower. Furthermore, standard ROC-AUC produces falsely optimistic scores due to the overwhelming number of true negatives in rare classes. Macro PR-AUC evaluates the area under the precision-recall curve across all recall thresholds independently for every class, providing a robust, threshold-invariant quality measure.

**Status: [Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide introduces Pipeline 2 (the continuous, data-centric Multimodal Knowledge Graph) and its runtime invocation policy: Conformal-Driven Lazy Execution. Querying a massive graph database on every routine inference call would destroy API throughput and inflate latency. Instead, unambiguous singleton predictions ($|\hat{C}(X)| = 1$) resolve on the Fast Hot-Path using INT8 ONNX models in $\leq 22.0$ ms. The MMKG and Vision-RAG pipeline are lazy-loaded exclusively when the Split Conformal Safety Gate detects epistemic uncertainty: severe classification ambiguity ($|\hat{C}(X)| > 3$) where sibling taxa cannot be separated within the 95% confidence bound, or complete novelty/OOD remnants ($|\hat{C}(X)| = 0$).

**Status: [Target Architecture / Roadmap]**

# Visual Pre-Processing

## Parent-Child Visual Chunking ($I_{\text{child}}$ vs $I_{\text{parent}}$)

### Slide Topic & Explanatory Focus

This slide details our visual entity chunking strategy designed to eliminate background clutter in field photography. Standard visual retrieval across uncropped citizen-science images introduces heavy noise from surrounding soil, rocks, overcast sky, or human hands holding the specimen. TaxonVision employs an upstream zero-shot detector (YOLO-World or OWL-ViT v2) to crop the primary organism ($I_{\text{child}}$), which is resized to $224 \times 224$ and passed to frozen BioCLIP-2 to compute dense morphological embeddings ($z_{\text{child}} \in \mathbb{R}^{768}$). Simultaneously, the uncropped parent scene context ($I_{\text{parent}}$) is retained for macro-habitat and environmental verification.

**Status: [Target Architecture / Roadmap]**

## Slide Topic & Explanatory Focus

This slide addresses the critical biological trap of convergent evolution—where completely unrelated species on different continents evolve near-identical morphologies (e.g., European wasps vs. North American cicada killers). Pure visual nearest-neighbor search frequently retrieves geographically impossible candidates. TaxonVision implements single-stage filtered HNSW traversal: spatial constraints (Haversine geodesic radius $R_{\max} \leq 500$ km) and temporal constraints ($\Delta_{\text{season}} = 1$ month) are evaluated as boolean predicates natively inside the C++ HNSW graph walk, pruning impossible species before nearest-neighbor calculation completes.

**Status: [Target Architecture / Roadmap]**

# Ensemble Ranking

## Multi-Modal Re-Ranking via Reciprocal Rank Fusion (RRF)

### Slide Topic & Explanatory Focus

This slide explores how TaxonVision synthesizes visual similarity and biological occurrence priors through Reciprocal Rank Fusion (RRF). After filtered HNSW retrieval yields candidate taxa, the system calculates a unified ranking score combining dense visual cosine similarity ($w_{\text{vis}} = 0.65$) with historical GBIF spatial-temporal occurrence density ($w_{\text{geo}} = 0.35$):

$$\text{RRF}(d) = \frac{w_{\text{vis}}}{k + r_{\text{vis}}(d)} + \frac{w_{\text{geo}}}{k + r_{\text{geo}}(d)}$$

This non-linear rank combination balances fine-grained visual evidence against ecological likelihood without requiring complex score normalization between divergent probability distributions.

**Status: [Target Architecture / Roadmap]**

## Slide Topic & Explanatory Focus

This slide details our multimodal reasoning interface for human taxonomists. When an ambiguous observation reaches triage, the query crop ($I_{\text{child}}$) and the top-3 re-ranked exemplar specimens are injected into a local Vision-Language Model (e.g., Qwen2-VL-2B). Constrained by formal grammar-based sampling, the VLM generates a structured JSON differential diagnosis card detailing distinguishing morphological traits (e.g., "dorsal stripe width", "antenna segment count"), ecological likelihoods, and CAM activation overlays. These diagnosis cards are rendered directly into Label Studio, transforming triage into rapid, high-precision expert verification.

**Status: [Target Architecture / Roadmap]**

# Taxonomic Agility

## Zero-Retraining Taxonomic Mutations via DuckDB

### Slide Topic & Explanatory Focus

This slide examines how TaxonVision handles dynamic changes in biological taxonomy without expensive GPU model retraining. In biological science, taxonomic committees frequently alter classifications: merging species (taxonomic lumps) or dividing a single species into multiple distinct species (taxonomic splits). Because visual vectors reside in HNSW while taxonomic relationships and observation metadata reside in DuckDB and Parquet, taxonomic lumps execute as $\mathcal{O}(1)$ relational updates with zero GPU re-embedding cost. Splits mark historical nodes as requiring re-annotation, routing them to Label Studio before re-indexing.

**Status: [Target Architecture / Roadmap]**

# Epistemic Safety

## The Illusion of Softmax Overconfidence

### The Danger of Raw Softmax

- **Mathematical Flaw:** Deep neural networks lack mathematical rigor in their confidence.
- **Exponentiation of Logits:** The softmax activation exponentiates the largest logit, frequently outputting confidence scores $> 0.95$ on completely erroneous predictions, unseen species, or corrupted inputs.
- **Catastrophic Real-World Impact:** In ecological monitoring and toxic species identification (e.g., distinguishing poisonous mushrooms from edible ones), blind reliance on raw softmax leads to fatal misclassifications.

### Engineering Rationale

- We cannot rely on heuristic thresholding on raw softmax scores for safety-critical biological systems. Rigorous mathematical calibration is required to explicitly bound inference risk.

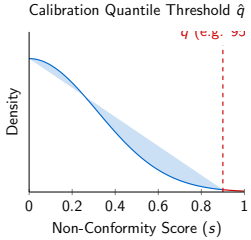
**Status:** **[Implemented & Verified]**

## Mathematical Foundation

- **Distribution-Free Guarantee:** Split Conformal Prediction ensures the true class label $y$ is contained within the prediction set $C(X)$ with user-defined probability $1 - \alpha$.
- **Coverage Bound:**

$$P(y \in C(X)) \geq 1 - \alpha$$

- **Calibration:** Uses a held-out calibration set to compute non-conformity scores ($s_i = 1 - \hat{f}(x_i)_{y_i}$) and determine the empirical quantile threshold $\hat{q}$.

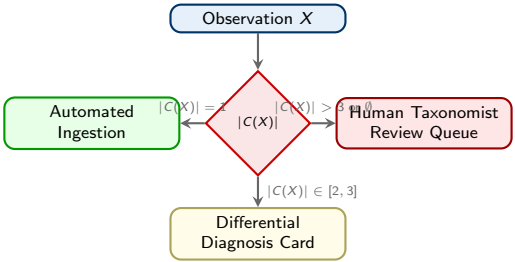



*Dynamic prediction sets scale from singletons (low ambiguity) to multi-class sets based on calibrated quantiles.*

**Status: [Implemented & Verified]**

## Epistemic Uncertainty Metric

- **Cardinality as a Signal:** The prediction set size $|C(X)|$ serves as an explicit, interpretable measure of epistemic uncertainty.
- **Operational Mechanics:** By measuring set size, we seamlessly integrate machine learning into human workflows, balancing autonomous efficiency with expert oversight to eliminate silent model errors.




**Status: [Implemented & Verified]**

## Pre-Classification Safety Gate

- **The Softmax Trap:** Probabilities normalized over known classes cannot reliably detect when an image does not belong to ANY known biological category (e.g., an indoor living room, vehicle, or macro photograph of rock mineral).
- **Energy-Based Scores:** Computes the negative Helmholtz free energy of the un-normalized logits:

$$E(x; f) = -T \cdot \log \sum_{i=1}^{K} e^{f_i(x)/T}$$

- **Engineering Rationale:** Observations with energy exceeding the calibrated threshold are immediately flagged as OOD prior to classification inference, shielding downstream pipelines from nonsensical data.

**Status: [Implemented & Verified]**

# Model Explainability

## Forward-Hooked Class Activation Mapping ($< 2$ ms)

### Visual Saliency & Interpretability

- **Validation by Taxonomists:** Ecological validation requires verifying that the model relies on genuine diagnostic morphological features (e.g., petal shape or antennae structure) rather than spurious background correlations (e.g., garden fences).
- **Forward-Hook Saliency (Eigen-CAM):** Standard Grad-CAM requires a backward pass that doubles latency. Our implementation utilizes PyTorch forward hooks on penultimate feature activations.
- **Engineering Rationale:** Generates class-discriminative visual saliency heatmaps in under 2 milliseconds without backward gradient overhead, ensuring real-time interpretability within low-latency service constraints.

**Status: [Implemented & Verified]**

# CI/CD Architecture

## Dual Workflow: GitHub Actions Cloud vs Jenkins Bare-Metal

### Slide Topic & Explanatory Focus

This slide examines our production CI/CD architecture, explaining the strategic division between cloud-hosted GitHub Actions and on-premise Jenkins CD. Public pull requests cannot be granted direct network or SSH access to private bare-metal GPU clusters without creating extreme security risks. Instead, GitHub Actions executes cloud-based static analysis gates (Ruff linting, Mypy typing, cyclomatic complexity, license compliance, OKF validation). Once PRs are verified and merged into `develop` or `main`, Jenkins on `hive-mind` triggers bare-metal deployment, container builds, and GPU integration testing within isolated Kubernetes pods.

**Status: [Implemented & Verified]**

# Code Quality

## Cognitive Complexity Gates: complexipy $\leq 15$ & Strict Typing

### Slide Topic & Explanatory Focus

This slide explores the software engineering principles enforced across our Python codebase. Machine learning codebases often degrade into unmaintainable "research spaghetti" filled with deeply nested loops, global state, and un-typed tensors. TaxonVision enforces strict automated gates: Ruff for formatting and linting, Mypy for static type verification, and `complexipy` enforcing a strict cognitive complexity limit of $\leq 15$ per function. Functions exceeding this threshold are rejected at pre-commit, forcing modular refactoring, helper extraction, and clean architecture.

**Status:** **[Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide covers production containerization using multi-stage Docker builds published to GitHub Container Registry (GHCR). Naïve Docker images package full compiler toolchains, package caches, and test fixtures, bloating images to $> 10$ GB. Our multi-stage build uses an isolated builder stage to resolve Pixi dependencies and compile extensions, copying only the minimal runtime artifacts into the final production image. This reduces final image sizes to $< 1.5$ GB, accelerates Kubernetes pod startup times, and minimizes the security attack surface.

**Status: [Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide details our bare-metal cluster management using k3s, a certified lightweight Kubernetes distribution optimized for edge and single-node enterprise clusters. We discuss how Jenkins dynamically provisions ephemeral agent pods inside the cluster to execute build pipelines, mounting local NVMe Rattler package caches to eliminate redundant network downloads. Once the pipeline job completes, the agent pod is automatically destroyed, ensuring complete environmental cleanliness without lingering processes or zombie CUDA allocations.

**Status:** **[Implemented & Verified]**

# Deployment Strategy

## Zero-Downtime Rolling Updates & Health Probes

### Slide Topic & Explanatory Focus

This slide illustrates our zero-downtime deployment strategy in Kubernetes. Updating an inference service must never drop active HTTP connections or return 502 Bad Gateway errors to clients. We configure Kubernetes Deployment manifests with readiness and liveness probes targeting FastAPI `/health` endpoints. When a new container image is deployed, Kubernetes waits for model weights to load and warm up in memory before switching incoming Traefik ingress traffic, terminating older replica pods only after new pods are fully serving traffic.

**Status:** **[Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide covers production observability and real-time telemetry. In MLOps, monitoring goes far beyond standard HTTP 200/500 error counts. Our FastAPI serving service exposes a `/metrics` endpoint scraped by Prometheus, capturing domain-specific metrics: p50/p95/p99 inference latency, GPU memory utilization, energy-based OOD detection rates, prediction set cardinality distribution, and human triage escalation ratios. Grafana dashboards visualize these metrics in real-time, enabling operators to detect system degradation before users notice.

**Status:** **[Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide explores statistical drift detection across high-dimensional feature spaces (`src/taxon_vision/observability/drift.py`). As seasons change, flora blooms, or new mobile cameras enter the field, input distributions drift away from training baselines. We contrast 1D marginal statistical tests (such as Kolmogorov-Smirnov or 1D Wasserstein-1 distance across individual feature projections) with full-manifold multivariate metrics like Maximum Mean Discrepancy (MMD) with RBF kernels. When drift exceeds calibrated statistical thresholds, automated alerts trigger the retraining pipeline.

**Status: [Implemented & Verified]**

## Slide Topic & Explanatory Focus

This slide presents our active learning subsystem: the BADGE (Batch Active learning by Diverse Gradient Embeddings) query strategy and human triage interface. When thousands of unlabelled field observations stream in, random sampling or pure uncertainty sampling yields redundant, low-diversity annotations. BADGE computes hypothetical gradient embeddings representing both epistemic uncertainty and feature diversity, selecting the most informative batch via $k$-MEANS++. These priority samples are pushed to Label Studio for expert verification, closing the feedback loop.

**Status: [Partially Implemented / In Progress]**

# Project Roadmap

## Implementation Status vs Architectural Roadmap

### Slide Topic & Explanatory Focus

This slide provides an honest, transparent breakdown of the current implementation status versus future architectural milestones. Phase 1 (Foundation: Pixi toolchain, DVC/S3 streaming, pre-commit gates, license compliance) and Phase 2 (Model Training & Infrastructure: frozen feature extraction, Class-Balanced Loss, Optuna tuning, conformal uncertainty sets, ONNX quantization, GHCR container builds, k3s manifests) are fully implemented and verified. Phase 3 (Continuous Intelligence: full Milvus Multimodal Knowledge Graph reasoning and end-to-end automated retraining webhooks) represents the target roadmap.

**Status: [Implemented & Verified]**

# Pedagogical Takeaways

## Classic MLOps Anti-Patterns & Engineering Fixes

### Slide Topic & Explanatory Focus

This slide summarizes the core pedagogical lessons for students transitioning into MLOps, contrasting common junior anti-patterns with enterprise architectural solutions. We contrast: (1) monolithic Jupyter notebook sprawl vs modular, typed Python packages with complexity limits; (2) hoarding local datasets vs zero-copy S3 streaming with DVC pointers; (3) uncalibrated softmax confidence vs mathematically guaranteed Split Conformal Prediction sets; and (4) unbudgeted cloud GPU spending vs disciplined bare-metal GPU time-slicing with strict memory ceilings.

**Status: [Implemented & Verified]**

## Slide Topic & Explanatory Focus

This final slide synthesizes the core philosophy of TaxonVision: building machine learning systems that are reproducible by default, mathematically safe under uncertainty, and economically sustainable on physical hardware. We emphasize that MLOps is not merely stringing together tools, but a rigorous systems engineering discipline that balances algorithmic precision, physical hardware constraints, operational telemetry, and human expert collaboration to solve critical real-world challenges like global biodiversity preservation.

**Status:** **[Implemented & Verified]**